Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java/Python

Département : Informatique FSSM
Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026

Plan

Chapitre 6 : Introduction à Python

- Présentation de Python
- Les variables et les types de valeurs
- Les structures de contrôle
- Les données composites

Présentation de Python

Historique et Définition

- Le langage de programmation Python a été créé en 1989 par Guido van Rossum
- La première version publique du langage est sortie en 1991.
- Il est multiplateforme. C'est-à-dire qu'il fonctionne sur de nombreux systèmes d'exploitation : Windows, Mac OS X, Linux, Android, iOS, depuis les mini-ordinateurs Raspberry Pi jusqu'aux supercalculateurs.
- Il est gratuit. Vous pouvez l'installer sur autant d'ordinateurs que vous voulez (même sur votre téléphone !).
- Un langage de “très haut niveau” → Il possède un haut niveau d’abstraction par rapport au langage machine.
- Un langage orienté objet à la base mais supporte également d’autres paradigmes comme la programmation procédurale et fonctionnelle.
- Un langage interprété → Il a besoin d’un programme nommé interpréteur pour convertir les instructions Python en un langage compréhensible par l’ordinateur.
- Modulable et extensible → on peut ajouter de nouvelles fonctions ou modules pour étendre les possibilités du langage.
- Deux versions cohabitent : Python 2 (obsolète) et Python 3

Environnement de travail

- Pour pouvoir utiliser Python, il faut installer un interpréteur.
- L'interpréteur Python est disponible en téléchargement gratuit sur le site officiel du langage, dans la rubrique téléchargement :

  https://www.python.org/downloads/

Vérification post-installation

- Lancer un shell PowerShell
- Vérifier la version installée :

  `Sous Windows : python -V`

  `Sous Mac : python3 --version`

- Appeler l'intérpréteur :

  `Sous Windows : python`

  `Sous Mac : python3`

```
oti@ilyasss-MacBook-Pro-2 ~ % python3 --version
Python 3.9.6
oti@ilyasss-MacBook-Pro-2 ~ % python3
Python 3.9.6 (default, May  7 2023, 23:32:44)
[Clang 14.0.3 (clang-1403.0.22.14.1)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print ("Hello dear ISI students")
Hello dear ISI students
>>>
```

- Afficher un premier message :

  `>>> print(" Hello dear ISI students!")`

Exécution de Python

- Deux options :

1. Ecrire directement les instructions Python dans l'interpréteur Python pour les exécuter immédiatement.

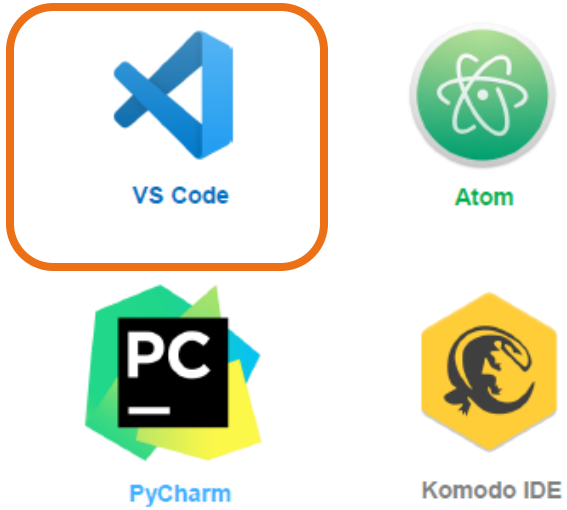
   Les trois caractères >>> constituent le signal d'invite, ou prompt principal, lequel vous indique que Python est prêt à exécuter une commande

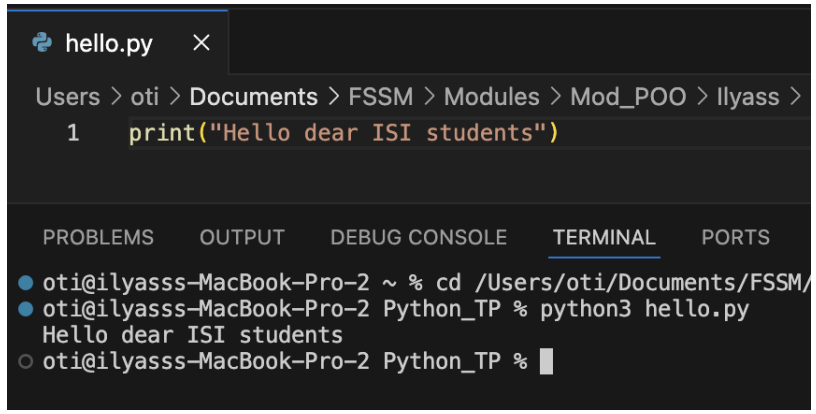
```
>>> print("Hello world!")
Hello world!
>>>
```

2. Créer des fichiers de script Python avec l'extension .py avec un éditeur de texte et les passer ensuite à l'interpréteur.

```
$ python test.py
Hello world!
```

- Plusieurs éditeurs : PyCharm, Microsoft VS Code, Brackets, Atom, Sublime Text, Komodo…





Syntaxe de base

- Pour ajouter un commentaire dans un code Python, on utilise le signe #.

```
1  # Votre premier commentaire en  Python.
2  print("Hello world!")
3
4  # D'autres commandes plus utiles pourraient suivre.
```

- En Python, l'indentation est utilisée pour définir des blocs de code, c'est-à-dire pour indiquer à l'interpréteur quelle instruction appartient à quelle autre. →l'ensemble des lignes indentées constitue un bloc d'instructions.

```
if embranchement == "vertébrés":
    if classe == "mammifères":
        if ordre == "carnivores":
            if famille == "félins":
                print "c'est peut-être un chat"
        print "c'est en tous cas un mammifère"
    elif classe == 'oiseaux':
        print "c'est peut-être un canari"
print"la classification des animaux est complexe"
```

Les variables et les types de valeurs Python

Variables et opérateur d'affectation

- Une variable est une zone de la mémoire de l'ordinateur dans laquelle une valeur est stockée.
- Aux yeux du programmeur, une variable est définie par un nom, alors que pour l'ordinateur, il s'agit en fait d'une adresse, c'est-à-dire d'une zone particulière de la mémoire.
- En Python, la déclaration d'une variable et son initialisation (c'est-à-dire la première valeur que l'on va stocker dedans) se font en même temps.

```
1  >>> x = 2
2  >>> x
3  2
```

Variables et opérateur d'affectation

- Python a « deviné » que la variable était un **entier**. On dit que Python est un langage au typage dynamique.
- Python a alloué (réservé) l'espace en mémoire pour y accueillir un entier. Chaque type de variable prend plus ou moins d'espace en mémoire. Python a aussi fait en sorte qu'on puisse retrouver la variable sous le nom x.
- Enfin, Python a assigné la valeur 2 à la variable x (affectation).

➔Dans d'autres langages (en C par exemple), il faut coder ces différentes étapes une par une. Python étant un langage dit de haut niveau, la simple instruction x = 2 a suffi à réaliser les 3 étapes en une fois !

- Lignes 2 et 3. L'interpréteur nous a permis de connaître le contenu de la variable juste en tapant son nom.

```
1  >>> x = 2
2  >>> x
3  2
```

Variables et opérateur d'affectation

- Sous Python, on peut assigner une valeur à plusieurs variables simultanément.
- On peut aussi effectuer des affectations parallèles à l'aide d'un seul opérateur

```
>>> x = y = 8
>>> x
8
>>> y
8
>>> x,y = 14.2 , 20
>>> x
14.2
>>> y
20
>>>
```

Affectations multiples

Affectations parallèles

Nommage

- Le nom des variables en Python peut être constitué de :
  - lettres minuscules (a à z)
  - lettres majuscules (A à Z)
  - nombres (0 à 9)
  - caractère souligné (_).
  - Vous ne pouvez pas utiliser d'espace dans un nom de variable.
- Par ailleurs, un nom de variable ne doit pas débuter par un chiffre et il n'est pas recommandé de le faire débuter par le caractère _ (sauf cas très particuliers).
- De plus, il faut absolument éviter d'utiliser un mot « réservé » par Python comme nom de variable (par exemple : print, range, for, from, etc.).
- Et, bien que possible avec Python 3, l'utilisation de caractères accentués dans les noms des variables est fortement **déconseillée**.
- Enfin, Python est sensible à la casse, ce qui signifie que les variables TesT, test et TEST sont différentes.

Affichage

- Pour afficher la valeur à l'écran, il existe deux possibilités. :

1 - La première consiste à entrer au clavier le nom de la variable, puis <Entrée>.

```
>>> a = 2
>>> a
2
>>> msg = "Quoi de neuf ?"
>>> msg
'Quoi de neuf ?'
>>>
```

2 - l'instruction print :

```
>>> print(msg)
Quoi de neuf ?
```

Affichage

- la fonction print() affiche l'argument qu'on lui passe entre parenthèses et un retour à ligne.
- La fonction print() peut également afficher le contenu d'une variable quel que soit son type. Par exemple, pour un entier :

```
1  >>> var = 3
2  >>> print(var)
3  3
```

- Il est également possible d'afficher le contenu de plusieurs variables

```
1  >>> x = 32
2  >>> nom = "John"
3  >>> print(nom, "a", x, "ans")
4  John a 32 ans
```

- A l'intérieur d'un programme, vous utiliserez toujours l'instruction print.

Saisie de données

- Pour permettre à l'utilisateur d'un programme de saisir la valeur d'une variable x, on utilise la fonction input(),
- ATTENTION : la variable saisie est toujours de type str. Pour la convertir en nombre, il faut utiliser la fonction int() ou la fonction float() :

```
x = int(input("saisir la valeur de x : "))
```

```
>>> x = input("saisir la valeur de x : ")
saisir la valeur de x : 12
>>> x
'12'
>>> type(x)
<class 'str'>
>>> x = int(x)
>>> x
12
>>> type(x)
<class 'int'>
```

Quelle est, à votre avis, l'utilité de la fonction type() ?

Saisie de données

- Exemple

```
#exemple de programme Python
x = float(input("la valeur de x : "))
y = float(input("la valeur de y : "))

print("x :",x)
print("y :",y)

test = (x + y) / 2

print("Résultat  :",test)
```

Les types de variables

Types Simples

- Nombre (integer/float/complex) : 976, 0.14, -9.99, complex(3,-4)
- Chaîne de caractères (str) : "hello"
- Booléen (boolean) : True, False
- Aucune valeur : None

Types Composites

- Liste : [1, 2, 3]
- Tuple : (1, 2, 3)
- Dictionnaire : { "Nom" : "Ali", "Age" : 29}
- Ensemble (Set) : { "Ali", 29, "Etudiant"}

Les nombres

- entiers (integer ou int)
- réels ou virgules flottantes (float)

```
>>> a, b, c = 1, 2, 5 # a,b et c seront de type int
>>> x, y, z = 22.9, 2., 0.12 # x,y et z seront de type float
```

Les nombres – Opérateurs arithmétiques

- Opérations sur les types numériques - opérateurs

Opération	Résultat
x + y	somme de x et y
x - y	différence de x et y
x * y	produit de x et y
x / y	quotient de x et y (division réelle)
x // y	quotient entier de x et y (division entière)
x % y	reste de x sur y (modulo)
abs(x)	valeur absolue de x
int(x)	x converti en nombre entier
float(x)	x converti en nombre à virgule flottante
x ** y	x à la puissance y

```
1  >>> 5 // 4
2  1
3  >>> 5 % 4
4  1
5  >>> 8 // 4
6  2
7  >>> 8 % 4
8  0
```

Les nombres – Opérateurs arithmétiques

- Opérations sur les types numériques - opérateurs combinés : permet d'effectuer une opération et une affectation en une seule étape,

Opérateur	Exemple	Raccourci pour
+=	x += 5	x = x + 5
-=	x -= 3	x = x – 3
*=	x *= 6	x = x * 6
//=	x //= 2	x = x // 2
**=	x**=4	x = x ** 4
%=	x%=5	x = x % 5

```
1   >>> i = 0
2   >>> i = i + 1
3   >>> i
4   1
5   >>> i += 1
6   >>> i
7   2
8   >>> i += 2
9   >>> i
10  4
```

Les chaines de caractères

- Python reconnaît certains types de variable automatiquement (entier, float). Par contre, pour une chaîne de caractères, il faut l'entourer de guillemets (doubles, simples, voire trois guillemets successifs doubles ou simples) afin d'indiquer à Python le début et la fin de la chaîne de caractères.
- Dans l'interpréteur, l'affichage direct du contenu d'une chaîne de caractères se fait avec des guillemets simples, quel que soit le type de guillemets utilisé pour définir la chaîne de caractères.

```
>>> a = "bonjour"
>>> a
'bonjour'
>>> b = 'salut'
>>> b
'salut'
>>> c = """girafe"""
>>> c
'girafe'
>>> d = '''lion'''
>>> d
'lion'
```

Les chaines de caractères

- Opérations sur les chaînes de caractères : Pour les chaînes de caractères, deux opérations sont possibles, l'addition et la multiplication :

```
1  >>> chaine = "Salut"
2  >>> chaine
3  'Salut'
4  >>> chaine + " Python"
5  'Salut Python'
6  >>> chaine * 3
7  'SalutSalutSalut'
```

Opérations

- Opérations illicites

```
>>> "toto" * 1.3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can't multiply sequence by non-int of type 'float'
>>> "toto" + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

- Notez que Python vous donne des informations dans son message d'erreur.

Exercice

- Essayez de prédire le résultat de chacune des instructions suivantes, puis vérifiez-le dans l'interpréteur Python :
  - `(1+2)**3`
  - `"Da" * 4`
  - `"Da" + 3`
  - `("Pa"+"La") * 2`
  - `("Da"*4) / 2`
  - `str(4) * int("3")`
  - `int("3") + float("3.2")`
  - `str(3) * float("3.2")`
  - `str(3/4) * 2`

Les Booléens

- Les Booléens sont basés sur la prise des décisions.
- Deux valeurs possibles : True (Vrai) ou False (faux)

```python
a = 6
b = 7
c = 42
print(1, a == 6)
print(2, a == 7)
print(3, a == 6 and b == 7)
print(4, a == 7 and b == 7)
print(5, not a == 7 and b == 7)
print(6, a == 7 or b == 7)
print(7, a == 7 or b == 6)
print(8, not (a == 7 and b == 6))
print(9, not a == 7 and b == 6)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
1 True
2 False
3 True
4 False
5 True
6 True
7 False
8 True
9 False
```

Comparaisons

```
>>> ch1 = "python"
>>> ch2 = "Python"
>>> ch1 == ch2
False
>>> ch2 = "python"
>>> ch1 == ch2
True
>>>
```

- les expressions booléennes sont le plus souvent constituées :

Les opérateurs de comparaison

Opérateur	Signification
<	strictement inférieur
<=	inférieur ou égal
>	strictement supérieur
>=	supérieur ou égal
==	égal
!=	different

opérateurs logiques

Opérateur	Signification
AND	Renvoie True si toutes les deux expressions sont évaluées à True
OR	Renvoie True si une des comparaisons vaut True
NOT	Renvoie True si la comparaison vaut False (et inversement)

Les comparaisons peuvent être enchaînées arbitrairement ; par exemple, x < y <= z est équivalent à x < y and y <= z.

Les structures de contrôle

Les structures de contrôle

- Les structures de contrôle sont les groupes d'instructions qui déterminent l'ordre dans lequel les actions sont effectuées dans le programme.

Séquence d'instructions

- Sauf mention explicite, les instructions d'un programme s'exécutent les unes après les autres, dans l'ordre où elles ont été écrites à l'intérieur du programme.
- Le « chemin » d'exécution est appelé un flux d'instructions, et les constructions qui le modifient sont appelées des instructions de contrôle de flux.
- Python exécute normalement les instructions de la première à la dernière, sauf lorsqu'il rencontre une instruction conditionnelle comme l'instruction «if». Une telle instruction va permettre au programme de suivre différents chemins suivant les circonstances.

Les tests :

Forme 1 : if

- La valeur de la condition sera interprétée en True ou False . Si la condition est correcte (évaluée à True) : le bloc d'instructions s'exécuté

```
if condition :
        bloc d'instructions
```

```
>>> a = 150
>>> if (a > 0):
...     print("nombre positif ")
...
```

Indentation obligatoire

Les tests :

Forme 2 : if … else

- Si la condition mentionnée après if est VRAI (True), on exécute le `bloc1`; si la condition est fausse, on exécute le `bloc2` d'instructions.

```
if condition :
        bloc 1 d'instructions
else :
        bloc 2 d'instructions
```

```
>>> a = 150
>>> if (a > 0):
...     print("nombre positif ")
... else:
...     print("nombre négatif ou nul")
...
```

Les tests :

Forme 3 : if … elif … else

- On peut faire mieux encore en utilisant aussi l'instruction elif (contraction de « else if »)

```
if condition :
        bloc 1 d'instructions
elif condition :
        bloc 2 d'instructions
else :
        bloc 3 d'instructions
```

```
>>> a = 150
>>> if (a > 0) :
...     print("nombre positif ")
... elif (a < 0) :
...     print("nombre négatif ")
... else :
...     print("nombre nul")
...
```

Blocs d'instructions

- Sous Python, toutes les instructions composées ont toujours la même structure : une ligne d'en-tête terminée par un double point, suivie d'une ou de plusieurs instructions indentées sous cette ligne d'en-tête.

```
Exemple :
Ligne d'en-tête:
   première instruction du bloc
   ... ...
   ... ...
   dernière instruction du bloc
```

- S'il y a plusieurs instructions indentées sous la ligne d'en-tête, elles doivent l'être exactement au même niveau (comptez un décalage de 4 caractères, par exemple). Ces instructions indentées constituent ce que nous appellerons désormais un bloc d'instructions.

Blocs d'instructions

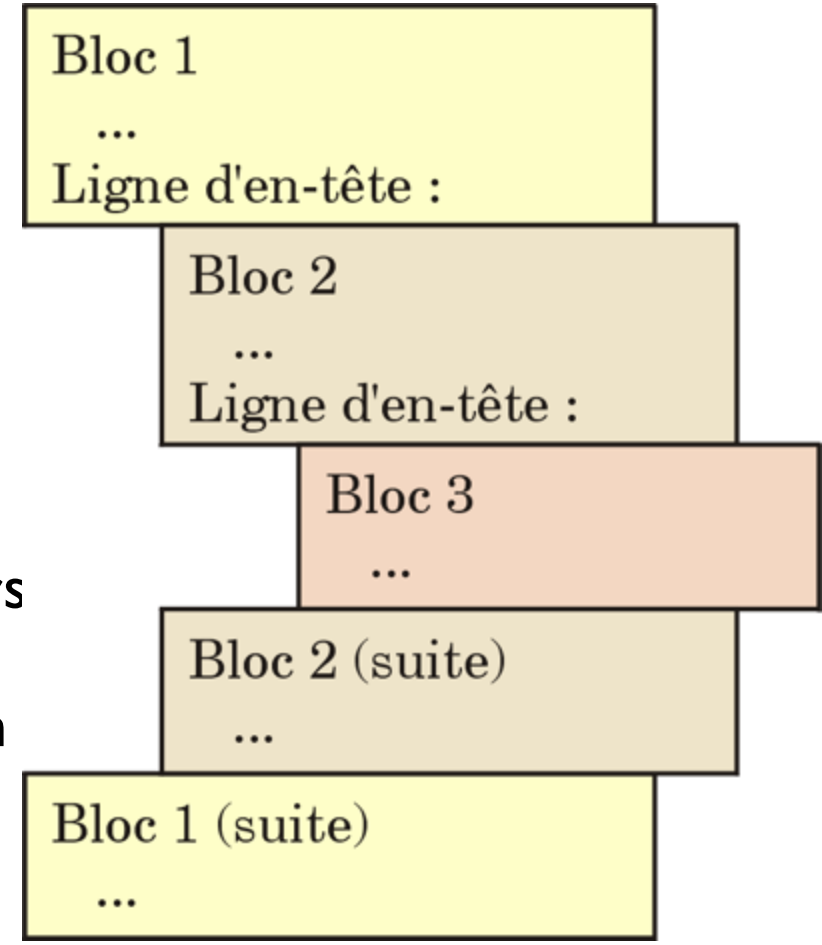
- Il est parfaitement possible d'imbriquer les unes dans les autres plusieurs instructions composées, de manière à réaliser des structures de décision complexes.

```
if embranchement == "vertébrés":                                  # 1
        if classe == "mammifères":                                # 2
                if ordre == "carnivores":                         # 3
                        if famille == "félins":                   # 4
                                print "c'est peut-être un chat"   # 5
                print "c'est en tous cas un mammifère"            # 6
        elif classe == 'oiseaux’:                                 # 7
                print "c'est peut-être un canari"                 # 8
print "la classification des animaux est complexe"                # 9
```

- Ce programme n'imprime la phrase « c'est peut-être un chat » que dans le cas où les quatre premières conditions testées sont vraies.
- Pour que la phrase « c'est peut-être un canari » soit affichée, il faut que la variable embranchement contienne « vertébrés », et que la variable classe contienne « oiseaux ».
- Quant à la phrase de la ligne 9, elle est affichée dans tous les cas, parce qu'elle fait partie du même bloc d'instructions que la ligne 1.

Blocs d'instructions

- Les blocs d'instructions sont toujours associés à une ligne d'en-tête contenant une instruction bien spécifique (if, elif, else, while, def, ...) se terminant par un double point.
- Les blocs sont délimités par l'indentation : toutes les lignes d'un même bloc doivent être indentées exactement de la même manière (c'est-à-dire décalées vers la droite d'un même nombre d'espaces). Le nombre d'espaces à utiliser pour l'indentation est quelconque, mais la plupart des programmeurs utilisent des multiples de 4.
- Vous pouvez aussi indenter à l'aide de tabulations, mais alors vous devrez faire très attention à ne pas utiliser tantôt des espaces, tantôt des tabulations pour indenter les lignes d'un même bloc.



Les boucles - while

- Python a deux types de structures répétitives : for et while
- La structure répétitive while permet d'effectuer une instruction ou des instructions (Bloc d'instructions) tant qu'une condition est satisfaite (évaluée à True)
- Ce qui signifie : tant que la condition est vraie, on exécute le bloc d'instructions.

```
while condition :
        Blocs instructions
```

Exemple 1

```
i=0
while i<10:
    print("bonjour")
```

Boucle infinie :
Bonjour s'affiche infiniment (sans arrêt)

Exemple 2

```
i=0
while i<10:
    print("bonjour")
    i+=1
```

Boucle correcte :
Bonjour s'affiche 10 fois.

Les boucles - for

- Dans la plupart des langages de programmation, la structure répétitive for permet de répéter des instructions un certain nombre de fois.
- En python, une boucle for est utilisée pour itérer sur une séquence (c'est-à-dire une liste, un tuple, un dictionnaire, un set, un intervalle de valeurs, ou une chaine de caractères). Autrement dit, elle permet de parcourir une séquence du premier au dernier élément.
- La boucle For peut parcourir une séquence de nombres en utilisant la fonction "range". La fonction range () renvoie une séquence de nombres, commençant par 0 par défaut et incrémentée de 1 (par défaut), et s'arrête avant un nombre spécifié. Syntaxe : range(start, stop, step) avec :
  - start (facultatif) : Un nombre **entier** spécifiant à quelle position commencer. La valeur par défaut est 0.
  - stop (Requis) : Un nombre **entier** spécifiant à quelle position s'arrêter (non inclus).
  - step (facultatif) : Un nombre **entier** spécifiant l'incrémentation. La valeur par défaut est 1

```
>>> for i in range(5):        # pour i allant de 0 à 4
...     print("bonjour")
```

Les boucles - for

- Exemples :

```
x = range(3, 20, 2)
for n in x:
  print(n)
```

```
>>> x = range(3, 20, 2)
>>> for n in x:
...   print(n)
...
...
3
5
7
9
11
13
15
17
19
```

```
x = range(3)
for n in x:
  print(n)
```

```
>>> x = range(3)
>>> for n in x:
...   print(n)
...
...
0
1
2
```

```
x = range(3,2)
for n in x:
  print(n)
```

```
>>> x = range(3,2)
>>> for n in x:
...   print(n)
...
...
>>>
```

Les boucles – continue et break

- continue est utilisée à l'intérieur d'une boucle pour ignorer le reste de l'itération actuelle.

```
for val in sequence:
    # code
    if condition:
        continue

    # code
```

```
while condition:
    # code
    if condition:
        continue
```

- break permet de stopper l'exécution d'une boucle lorsqu'une certaine condition est vérifiée.

```
for val in sequence:
    # code
    if condition:
        break

    # code
```

```
while condition:
    # code
    if condition:
        break

    # code
```

Les deux instructions sont généralement dans une condition de type if.

Les boucles – continue et break

- Exemples : continue

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
>>> for i in range(5):
...     if i == 3:
...         continue
...     print(i)
...
...
0
1
2
4
```

```
num = 0
while num < 10:
    num += 1
    if (num % 2) == 0:
        continue
    print(num)
```

```
>>> num = 0
>>> while num < 10:
...     num += 1
...     if (num % 2) == 0:
...         continue
...     print(num)
...
1
3
5
7
9
```

Les boucles – continue et break

- Exemples : break

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

```
>>> for i in range(5):
...     if i == 3:
...         break
...     print(i)
...
...
0
1
2
```

```
i = 1
while i <= 10:
    print('6 * ‘, i, '=',6 * i)
    if i >= 5:
        break
    i = i + 1
```

```
>>> i=1
>>> while i <= 10:
...     print('6 * ',i, '=',6 * i)
...     if i >= 5:
...         break
...     i = i + 1
...
...
6 *  1 = 6
6 *  2 = 12
6 *  3 = 18
6 *  4 = 24
6 *  5 = 30
>>>
```

Exercices

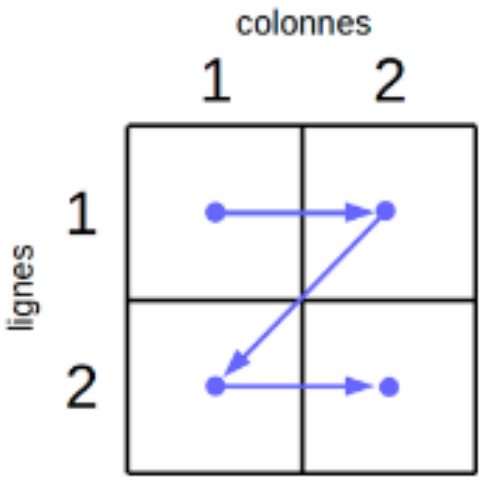
- Créez un script qui dessine un triangle comme celui-ci :

```
1   *
2   **
3   ***
4   ****
5   *****
6   ******
7   *******
8   ********
9   *********
10  **********
```

Exercices

- Imaginons que l'on souhaite parcourir tous les éléments d'une matrice carrée, c'est-à-dire d'une matrice qui est constituée d'autant de lignes que de colonnes.
- Créez un script qui parcourt chaque élément de la matrice et qui affiche le numéro de ligne et de colonne uniquement avec des boucles **for**.
- Pour une matrice de dimensions 2 X 2, le schéma de la figure ci-dessous vous indique comment parcourir une telle matrice. L'affichage attendu est :

```
ligne colonne
   1      1
   1      2
   2      1
   2      2
```



Les données composites

Les données composites

- Une donnée de type composite est une entité qui rassemble dans une seule structure un ensemble d'entités plus simples.
- Les types de données composites sont constitués d'autres types de données,
- Types Composites : Liste, Tuple, Dictionnaire, Ensemble (Set)

Les listes

- Une liste est une structure de données qui contient une série de valeurs.
- Python autorise la construction de liste contenant des valeurs de types différents.
- Une liste est déclarée par une série de valeurs séparées par des virgules, et le tout encadré par des crochets.

```
>>> animaux = ["girafe", "tigre", "singe", "souris"]
>>> tailles = [5, 2.5, 1.75, 0.15]
>>> mixte = ["girafe", 5, "souris", 0.15]
>>> animaux
['girafe', 'tigre', 'singe', 'souris']
>>> tailles
[5, 2.5, 1.75, 0.15]
>>> mixte
['girafe', 5, 'souris', 0.15]
>>>
```

Les listes

- Les listes sont des séquences ordonnées, c'est-à-dire des collections d'objets. On peut accéder à chacun d'entre eux individuellement si l'on connaît son indice (index) dans la liste.

```
liste          : ["girafe", "tigre", "singe", "souris"]
indice positif :        0        1        2        3
indice négatif :       -4       -3       -2       -1
```

```
>>> animaux = ["girafe", "tigre", "singe", "souris"]
>>> animaux[-2]
'singe'
>>> animaux[0]
'girafe'
>>> animaux[3]
'souris'
>>>
```

Les listes

- Tranches : Un autre avantage des listes est la possibilité de sélectionner une partie d'une liste en utilisant un indiçage construit sur le modèle [m:n+1] pour récupérer tous les éléments, du m-ième au n-ième (de l'élément m inclus à l'élément n+1 exclu). On dit alors qu'on récupère une tranche de la liste.

```
>>> animaux = ["girafe", "tigre", "singe", "souris"]
>>> animaux[0:2]
['girafe', 'tigre']
>>> animaux[0:3]
['girafe', 'tigre', 'singe']
>>> animaux[1:]
['tigre', 'singe', 'souris']
>>> animaux[:1]
['girafe']
>>> animaux[:]
['girafe', 'tigre', 'singe', 'souris']
>>> animaux[1:-1]
['tigre', 'singe']
>>> animaux[-3:]
['tigre', 'singe', 'souris']
```

Les listes

- Tranches : On peut aussi préciser le pas en ajoutant un symbole deux-points supplémentaire et en indiquant le pas par un entier.

```
>>> animaux = ["girafe", "tigre", "singe", "souris"]
>>> animaux[0:3:2]
['girafe', 'singe']
>>> x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> x
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> x[::1]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> x[::2]
[0, 2, 4, 6, 8]
>>> x[::3]
[0, 3, 6, 9]
>>> x[0:9:3]
[0, 3, 6]
>>> x[1:6:3]
[1, 4]
```

Les listes

- Opération sur les listes : Tout comme les chaînes de caractères, les listes supportent l'opérateur + de concaténation, ainsi que l'opérateur * pour la duplication.

```
>>> ani1 = ["girafe", "tigre"]
>>> ani2 = ["singe", "souris"]
>>> ani1 + ani2
['girafe', 'tigre', 'singe', 'souris']
>>> ani1 * 3
['girafe', 'tigre', 'girafe', 'tigre', 'girafe', 'tigre']
>>>
```

Les listes

- Une liste Python peut être modifiée. On dit que les listes sont mutables.
- 4 types de mutations :
  - Ajout : l'opérateur de concaténation + ou la méthode append(),

    La méthode append() ajoute un seul élément à la fin d'une liste;
  - Modification : par substitution;
  - Insertion; La méthode insert() insère un objet dans une liste à un indice déterminé;
  - Suppression. L'instruction del supprime un élément d'une liste à un indice déterminé

```
>>> a = [1, 2, 3]
>>> a
    [1, 2, 3]
>>> a = a + [5] # Ajout par addition
>>> a
    [1, 2, 3, 5]
>>> a.append(10) # Ajout par la méthode append()
>>> a
    [1, 2, 3, 5, 10]
>>> a[2] = 15 # Modification
>>> a
    [1, 2, 15, 5, 10]
>>> a.insert(3, -24) # Insertion
>>> a
    [1, 2, 15, -24, 5, 10]
>>> del a[3] # Suppression par l'instruction del
>>> a
    [1, 2, 15, 5, 10]
```

listes imbriquées

- Une liste peut contenir n'importe quel objet, même une autre liste, qui à son tour peut contenir des sous-listes elles-mêmes, etc. Il s'agit de la liste imbriquée. → listes imbriquées : listes de listes.

```
>>> enclos1 = ["girafe", 4]
>>> enclos2 = ["tigre", 2]
>>> enclos3 = ["singe", 5]
>>> zoo = [enclos1, enclos2, enclos3]
>>> zoo
[['girafe', 4], ['tigre', 2], ['singe', 5]]
>>> zoo[1]
['tigre', 2]
>>> zoo[1][0]
'tigre'
>>> zoo[1][1]
2
>>> zoo[1][1] = 3
>>> zoo
[['girafe', 4], ['tigre', 3], ['singe', 5]]
```

listes imbriquées

- On peut utiliser toutes les méthodes déjà vu qui facilitent les traitements classiques dans les listes.

```
>>> L = ['a', ['b', 'c'], 'd']
>>> L[1].append('xx')
>>> L
['a', ['b', 'c', 'xx'], 'd']
```

```
>>> L = ['a', ['b', 'c'], 'd']
>>> L[1].insert(0,'xx')
>>> L
['a', ['xx', 'b', 'c'], 'd']
```

```
>>> L = ['a', ['bb', 'cc'], 'd']
>>> del   L[1][1]
>>> L
['a', ['bb'], 'd']
```

Les listes - Méthodes

- `len(nomListe)` :Affiche la taille de la liste
- `sum(nomListe)` : Calcule la somme des éléments de la liste
- `max(nomListe)` : Cherche la valeur maximale dans la liste
- `min(nomListe)` : Cherche la valeur minimale dans la liste
- `nomListe.remove(Element)` : Retire un élément de la liste
- `nomListe.sort()` :Trie la liste
- `nomListe.pop()` : Retire le dernier élément de la liste

```
>>> x = [33,24,19,46,38]
>>> len(x)
5
>>> sum(x)
160
>>> max(x)
46
>>> min(x)
19
>>> x.remove(19)
>>> x
[33, 24, 46, 38]
>>> x.sort()
>>> x
[24, 33, 38, 46]
>>> x.pop()
46
>>> x
[24, 33, 38]
```

Exercices

- Exercice 1 :

Constituez une liste semaine contenant les 7 jours de la semaine.

1. À partir de cette liste, comment récupérez-vous seulement les 5 premiers jours de la semaine d'une part, et ceux du week-end d'autre part ? Utilisez pour cela l'indiçage.
2. Cherchez un autre moyen pour arriver au même résultat (en utilisant un autre indiçage).
3. Trouvez deux manières pour accéder au dernier jour de la semaine.
4. Inversez les jours de la semaine en une commande

- Exercice 2 : Remplir une liste avec 10 entiers donnés par l'utilisateur .Puis, le programme trouve le plus grand et le plus petit de ces nombres.
- Exercice 3 : Proposer un programme qui demande un nombre entier x entre 1 et 10 jusqu'à ce que la réponse convienne. Puis, le programme remplit une liste L avec les multiplicateurs de x qui sont entre 0 et 100.On affiche alors la liste.

Exercice 2

```
>>> jours = ['lun','mar','mer','jeu','ven','sam','dim']
>>> jours[0:5]   # 5 premiers jours
>>> jours[:5]    # 5 premiers jours
>>> jours[5:7]   # week-end
>>> jours[-2:]   # week-end
>>> jours[len(jours)-1]      # dernier jour
>>> jours[-1]                # dernier jour
>>> jours.reverse()          # inverser une liste
>>> jours
```

Exercice 2

```
l=[]
#Remplissage
for i in range(10):  l.append(int(input("Donner un nombre:")))
#Recherche le pp et pg
#1- initialisation
min=l[0]
max=l[0]
#2 -recherche dans le reste
for elt in l:
   if min>elt:
     min=elt
   if max<elt:
     max=elt
#3 affichage
print("Le plus petit :",min)
print("Le plus grand:",max)
print(l)
```

Exercice 3

```
while True :  x=int(input("Donner un entier entre 1 et 10:"))
   if x>10 or x<=0:
     print("Essayer une autre fois:")
   else:
      print("Bravo!!")
       break
l=[]
for i in range(0,100,x):
     l.append(i)

"""for i in range (0,100):
    m=x*i
    if m>=100:
        break
    l.append(m)
   """
print(l)
```

Les chaines de caractères

Chaînes de caractères ↔ Données composites

- Une chaîne de caractères est une donnée composite de type string str qui est une suite d'entités (ou items) plus simples, ➔ les caractères.
- On peut considérer une chaîne de caractères comme une séquence ordonnée de variables qui contiennent chacune un caractère. Les caractères sont indexés à partir de 0.
- Si on crée une variable de type str avec l'affectation chaine = 'Hello' , on obtient sa longueur avec len(chaine) et on accède au caractère d'indice i par son nom qui est chaine[i].
- Le type str est un type de variable non mutable.

Les chaines de caractères

Exemple

On observe que :

- on peut désigner un caractère directement par son indice ;
- on peut compter à partir de la fin ;
- on peut extraire des tranches;
- il n'est pas possible de changer une partie d'une chaîne.

```
>>> phrase = 'Python est un super langage de programmation'
>>> phrase[0:6]
'Python'
>>> phrase[0]
'p'
>>> phrase[-10]
'g'
>>> phrase[-10:]
'grammation'
>>> phrase[7:]
'est un super langage de programmation'
>>> phrase[:7]
'Python '
>>> phrase[0] = "p"
Traceback (most recent call last):
  File "<pyshell#55>", line 1, in <module>
    phrase[0] = "p"
TypeError: 'str' object does not support item assignment
>>> chaine = phrase[0:7] + 'v3'
>>> chaine
'Python v3'
```

exercice

1. Écrivez un script qui détermine si une chaîne contient ou non le caractère «a».
2. Écrivez un script qui compte le nombre d'occurrences du caractère «e» dans une chaîne.

Les tuples

- Les tuples sont des objets séquentiels correspondant aux listes (itérables, ordonnés et indexables) mais ils sont toutefois non modifiables.
- Les tuples sont un autre type séquentiel de données.
- Un tuple est une donnée immuable à la différence d'une liste qui est altérable. Cela signifie qu'on ne va pas pouvoir modifier les valeurs d'un tuple après sa création.
- L'intérêt des tuples par rapport aux listes réside dans leur immutabilité
- Pratiquement, on utilise les parenthèses au lieu des crochets pour les créer :

```
>>> t = (1, 2, 3)
>>> t
(1, 2, 3)
>>> type(t)
<class 'tuple'>
>>> t[2]
3
>>> t[0:2]
(1, 2)
>>> t[2] = 15
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    t[2] = 15
TypeError: 'tuple' object does not support item assignment
>>>
```

Les tuples

- Les opérateurs + et * fonctionnent comme pour les listes (concaténation et duplication) :

```
>>> (1, 2) + (3, 4)
(1, 2, 3, 4)
>>> (1, 2) * 4
(1, 2, 1, 2, 1, 2, 1, 2)
>>>
```

Les dictionnaires

- Les dictionnaires sont un autre type séquentiel de données.
- Leur différence réside dans leur façon d'indexer les valeurs.
- On a la liberté de choisir nous mêmes nos clefs (ou index ou indice) et attribuer la clef de notre choix à chaque valeur.
- Les valeurs d'un dictionnaire ne sont pas ordonnées à la différence des valeurs d'une séquence.
- On accède aux valeurs d'un dictionnaire par des clés

```
>>> ani1 = {}
>>> ani1["nom"] = "girafe"
>>> ani1["taille"] = 5.0
>>> ani1["poids"] = 1100
>>> ani1
{'nom': 'girafe', 'taille': 5.0, 'poids': 1100}
>>>
```

Les dictionnaires : Itération sur les clés

- Si on souhaite voir toutes les associations clés / valeurs, on peut itérer sur un dictionnaire de la manière suivante :

```
>>> ani2 = {'nom': 'singe', 'poids': 70, 'taille': 1.75}
>>> for key in ani2:
...     print(key, ani2[key])
...
...
nom singe
poids 70
taille 1.75
>>>
```

Les dictionnaires

```
>>> d = {"nom": "Ali", "age":30, "sport":["Football", "Tennis"]}
>>> d["nom"] #Afficher les éléments du dictionnaire
'Ali'
>>> d["sport"] #Afficher les éléments du dictionnaire
['Football', 'Tennis']
>>> d["email"] = "ali123@est.ma" #Ajouter un élément au dictionnaire
>>> d["age"] = 40 #Modifier un élément du dictionnaire
>>> d
{'nom': 'Ali', 'age': 40, 'sport': ['Football', 'Tennis'], 'email': 'ali123@est.ma'}
>>> del d["email"] #Supprimer un élément du dictionnaire
>>> d
{'nom': 'Ali', 'age': 40, 'sport': ['Football', 'Tennis']}
>>>
```

Les ensembles (Sets)

- Les ensembles sont un autre type de données composites.
- Un ensemble est une collection d'éléments modifiables, non ordonnée, sans index et qui ne peut pas posséder l'élément dupliqué.

```
>>> s = {4, 5, 5, 12}
>>> s
{12, 4, 5}
>>> type(s)
<class 'set'>
>>>
```

Les ensembles (Sets)

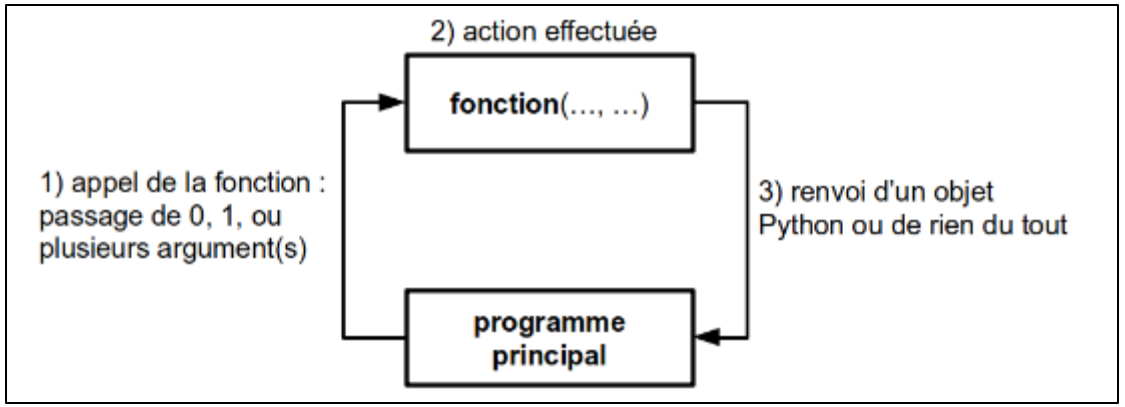
- La fonction interne à Python set() convertit un objet itérable passé en argument en un nouveau set (opération de casting)

```
>>> set([1, 2, 4, 1])
{1, 2, 4}
>>> set((2, 2, 2, 1))
{1, 2}
>>> set(range(5))
{0, 1, 2, 3, 4}
>>> set({"clé_1": 1, "clé_2": 2})
{'clé_2', 'clé_1'}
>>> set(["ti", "to", "to"])
{'ti', 'to'}
>>> set("Python version 3")
{'n', 'e', 'h', 'v', 'P', 'o', 's', 'r', 'i', '3', ' ', 'y', 't'}
>>>
```

Les fonctions

Fonctions : Principe et généralités

- Les fonctions permettent de décomposer un programme complexe en une série de sous-programmes plus simples, lesquels peuvent à leur tour être décomposés eux-mêmes en fragments plus petits, et ainsi de suite…
  - Par exemple si nous disposons d'une fonction capable de calculer une racine carrée, nous pouvons l'utiliser un peu partout dans nos programmes sans avoir à la réécrire à chaque fois.
- Vous connaissez déjà certaines fonctions Python. Par exemple range() ou len(). Pour l'instant, une fonction est à vos yeux une sorte de « boîte noire » :
  - À laquelle vous passez aucune, une ou plusieurs variable(s) entre parenthèses. Ces variables sont appelées arguments.
  - Qui effectue une action.
  - Et qui renvoie un objet Python ou rien du tout



Fonctions : Déclaration d'une fonction

- Pour définir une fonction, Python utilise le mot-clé `def`. Si on souhaite que la fonction renvoie quelque chose, il faut utiliser le mot-clé `return`.

```
def   nomFonction(parametre1,parametre2,…) :
            instructions
            return  valeur_de_retour
```

- l'indentation de ce bloc d'instructions (qu'on appelle le corps de la fonction) est obligatoire.
- valeur_de_retour = récupérable dans une variable
- Exemple :

```
>>> def carre(x):
...     return x**2
...
>>> print(carre(2))
4
```

Fonctions : Déclaration d'une fonction

Fonction sans paramètres sans retour

```
def myFunction() :
    instruction1
    instruction2
    …

> myFunction()
```

Fonction sans paramètres avec retour

```
def myFunction() :
    instruction1
    instruction2
    …
    return valeur

> var = myFunction()
```

Fonction avec paramètres sans retour

```
def myFunction(p1, p2, …) :
    instruction1
    instruction2
    …

> myFunction(valeur1, valeur2, …)
```

Fonction avec paramètres avec retour

```
def myFunction(p1, p2, …) :
    instruction1
    instruction2
    …
    return valeur

> retour = myFunction(valeur1, valeur2, …)
```

Fonctions : Déclaration d'une fonction

Fonction sans paramètres sans retour

```
>>> def hello():
...     print("bonjour")
...
>>> hello()
bonjour
```

Fonction sans paramètres avec retour

```
def filiere() :
... return "ISI"
...
>>> filiere()
'ISI'
```

Fonction avec paramètres sans retour

```
>>> def bonjour(p) :
... print("bonjour ", p)
...
>>> bonjour("mohamed" )
bonjour mohamed
```

Fonction avec paramètres avec retour

```
>>> def fois(x, y):
...     return x*y
...
>>> fois(2, 3)
6
>>> prod = fois(2, 3)
>>> prod
6
```

Fonctions : La portée (scope)

Portée

Le terme de « portée des variables » sert à désigner les différents espaces dans le script dans lesquels une variable est accessible, c'est-à-dire utilisable.

Portée globale

Une variable est dite globale lorsqu'elle est créée dans le programme principal. Elle sera visible partout dans le programme.

Portée locale

Une variable est dite locale lorsqu'elle est créée dans une fonction. Elles ne peuvent être utilisées qu'à l'intérieur de la fonction qui les a définies.

Fonctions : La portée (scope)

```
>>> x=10
>>> def afficher():
...     print(x)
...
...
>>> afficher()
10
```

Déclarer et afficher une variable globale

```
>>> def afficher():
...     x=20
...     print(x)
...
...
>>> afficher()
20
```

Déclarer et afficher une variable locale

```
>>> x = 10
>>> def portee():
...     global x
...     x=5
...
...
>>> x
10
>>> portee()
>>> x
5
```

Modifier une variable globale depuis une fonction

Fonctions : Fonctions prédéfinies

		Built-in Functions		
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

On peut découvrir toutes les fonctions built-in ici.

Exercices

- **Exercice 1 :**

1. Proposer une fonction en python qui permet de calculer la somme des éléments d'une liste
2. Utiliser cette fonction pour calculer la somme des éléments d'une liste L dont les éléments sont générés aléatoirement.

- **Exercice 2 :**

1. Proposer une fonction qui construit une liste les diviseurs d'un nombre la fonction doit retourner la liste construite.
2. tester la fonction avec un programme de test.

- **Exercice 3 : Utilisation des fonctions récursives**
- Une fonction récursive est une fonction qui s'appelle elle-même, toutefois, il faut faire attention à la condition d'arrêt sinon on risque d'exécuter la fonction à l'infinie

1. Proposer une fonction qui calcule la factorielle d'un nombre
2. Utiliser la fonction pour calculer la somme des factorielles des 100 premiers nombres positifs

Les modules standard et paquets Python

Définition d'un module

- Module Python : On appelle "module" tout fichier constitué de code Python (c'est-à-dire tout fichier avec l'extension .py) importé dans un autre fichier ou script.
- Il y a trois types de modules Python :
  - Les modules standards qui ne font pas partie du langage en soi mais sont intégrés automatiquement par Python ;
  - Les modules développés par des développeurs externes qu'on va pouvoir utiliser ;
  - Les modules qu'on va développer nous mêmes.
- Pour utiliser un module, on utilise la commande import.

Importation de modules

- Exemple :

```
>>> import random                    # 1
>>> random.randint(0, 10)            # 2
4                                    # 3
```

- Ligne 1, l'instruction import donne accès à toutes les fonctions du module `random`.
- Ensuite, ligne 2, nous utilisons la fonction `randint(0, 10)` du module `random`. Cette fonction renvoie un nombre entier tiré aléatoirement entre 0 inclus et 10 inclus.

Quelques modules courants

- math : fonctions et constantes mathématiques de base (sin, cos, exp, pi…).
- sys : interaction avec l'interpréteur Python, passage d'arguments.
- os : dialogue avec le système d'exploitation.
- random : génération de nombres aléatoires.
- time : accès à l'heure de l'ordinateur et aux fonctions gérant le temps.
- …
- Pour la liste complète, reportez-vous à la page des modules sur le site de Python.

Création de modules

- En Python, la création d'un module est très simple. Il suffit d'écrire un ensemble de fonctions (et/ou de constantes) dans un fichier, puis d'enregistrer ce dernier avec une extension .py (comme n'importe quel script Python).

message.py

```
DATE = 16092008
def bonjour(nom):
    return "Bonjour " + nom
def ciao(nom):
    return "Ciao " + nom
def hello(nom):
    return "Hello " + nom
```

Le chargement du module se fait avec la commande import message. Notez que le fichier est bien enregistré avec une extension .py et pourtant on ne la précise pas lorsqu'on importe le module. Ensuite, on peut utiliser les fonctions comme avec un module classique.

```
>>> import message
>>> message.hello("Mohamed")
'Hello Mohamed'
>>> message.ciao("Ali")
'Ciao Ali'
>>> message.bonjour("Monsieur")
'Bonjour Monsieur'
>>> message.DATE
16092008
```

Merci de votre attention

Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java/Python

Département : Informatique FSSM
Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026